



Steam: A deconstruction of a virtual video-game store

Juan Pablo Borja Espitia

Universidad distrital Francisco José de Caldas, Bogotá D.C, Colombia
Jpborjae@udistrital.edu.co

Introduction

Steam, developed by Valve Corporation in 2003, evolved from a game distribution service into one of the largest digital gaming platforms. This research analyzes its **database architecture, API design, and business model**, focusing on how it manages millions of users and game transactions. [1]



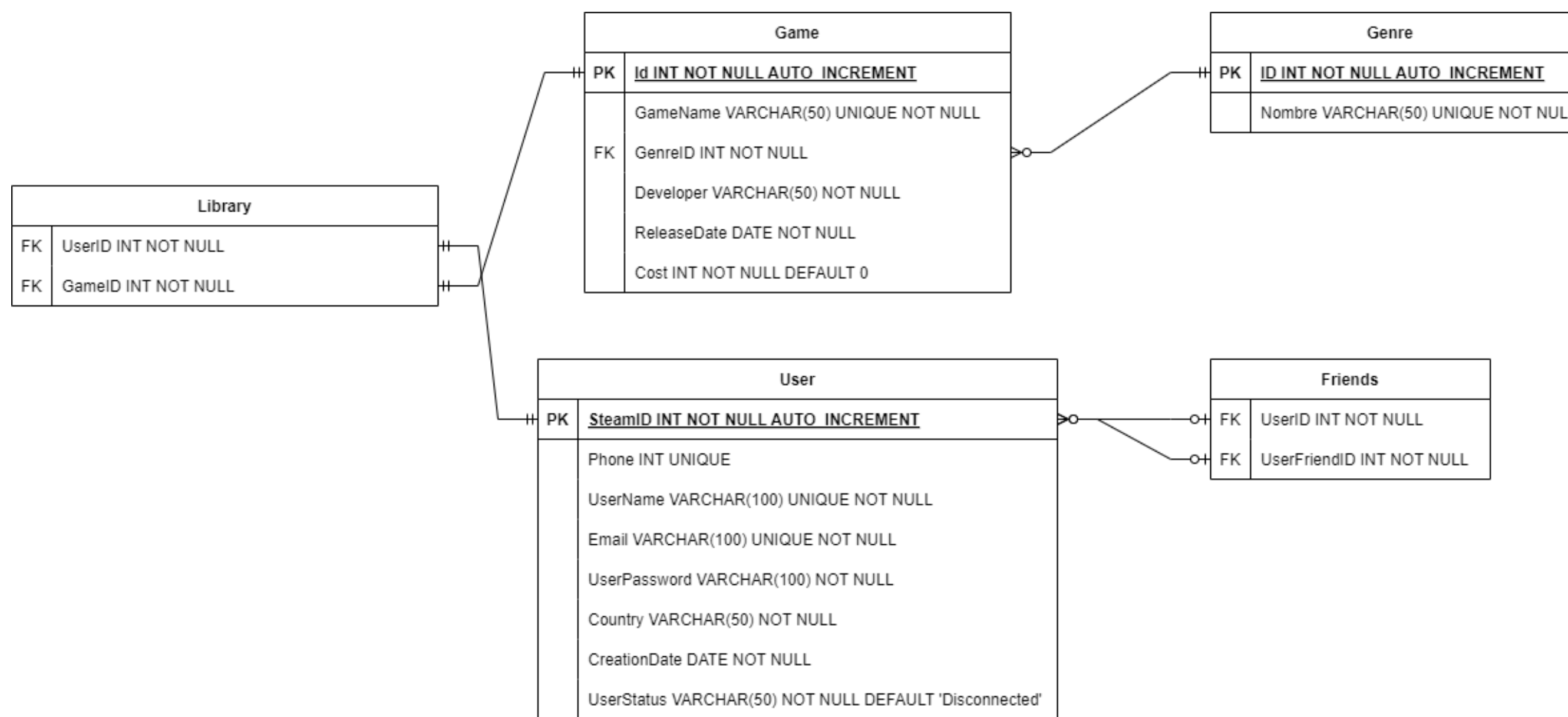
ERD Model

Entities:

- **User:** Stores user-related information, including SteamID, username, email, phone number, password, country, account creation date, and online status.
- **Friends:** Represents the friendship relationships between users, storing pairs of UserIDs with foreign key constraints enforcing cascading deletion.
- **Genre:** Defines game genres, storing a unique identifier and name for each genre.
- **Game:** Stores details about games, including the game name, genre, developer, release date, and cost. References the Genre table to associate each game with a genre.
- **Library:** Represents the Steam library, linking users to the games they own. Uses foreign keys to enforce relationships with the User and Game tables, ensuring cascading deletions.

Constraints

- **Primary Keys:**
 - **User:** SteamID (AUTO-INCREMENT)
 - **Friends:** Composite key (UserID, UserFriendID)
 - **Genre:** ID (AUTO-INCREMENT)
 - **Game:** Id (AUTO-INCREMENT)
 - **SteamLibrary:** Composite key (UserID, GameID)
- **Foreign Keys:**
 - **Friends:** UserID and UserFriendID reference **User(SteamID)**, with ON DELETE CASCADE.
 - **Game:** GenreID references **Genre(ID)**.
 - **SteamLibrary:**
 - * UserID references **User(SteamID)**, with ON DELETE CASCADE.
 - * GameID references **Game(Id)**, with ON DELETE CASCADE.
- **Unique Constraints:**
 - **User:** UserName, Email, and Phone must be unique.
 - **Genre:** Nombre must be unique.
- **Not Null Constraints:**
 - **User:** SteamID, UserName, Email, UserPassword, CreationDate, and UserStatus must not be null.
 - **Game:** Id, GameName, GenreID, Developer, ReleaseDate, and Cost must not be null.
 - **Genre:** ID and Nombre must not be null.
- **Default Values:**
 - **User:** UserStatus defaults to 'Disconnected'.
 - **Game:** Cost defaults to 0.



Structure

The project is organized into a modular structure to ensure **scalability, maintainability, and efficient development**. Below is an overview of the key directories and their roles:

- **/connections/** – Manages database connections, including the **db.connection.py** file for establishing communication with MySQL.
- **/crudOperations/** – Defines CRUD (Create, Read, Update, Delete) operations for different entities such as users, games, friends, and libraries.
- **/DataAccessObjects/** – Serves as the data abstraction layer, handling structured interactions with the database. It includes files for managing user data, game data, genre data, and more.

- **/db_init/** – Contains scripts for database initialization, including **init.sql** for schema creation and **init-script.sh** for setup automation.
- **/services/** – Implements FastAPI route handlers for different entities like users, games, friends, and libraries, providing a structured API endpoint architecture.
- **main.py** – The main entry point for the FastAPI application, handling API initialization and request routing.
- **Dockerfile & docker-compose.yml** – Define the containerized deployment environment, enabling easy setup and scalability using Docker.
- **requirements.txt** – Lists the Python dependencies required to run the project.

RESTful API Implementation

The project implements a **RESTful API** to facilitate structured and efficient interactions between clients and the database. The API is developed using **FastAPI**, a modern web framework for building APIs in Python, known for its high performance and automatic OpenAPI documentation.

5.1 Design and Implementation

The API follows RESTful principles, ensuring a standardized approach to handling resources. Each entity in the database (**User**, **Game**, **Friends**, **Genre**, and **Library**) is mapped to corresponding API endpoints, providing CRUD (Create, Read, Update, Delete) functionality.

- **FastAPI:** Chosen for its asynchronous support, automatic request validation, and easy integration with SQL databases.
- **MySQL Database:** Used as the primary data store, managed using SQLAlchemy for ORM-based interactions.
- **Docker and Docker Compose:** Ensures a consistent deployment environment and manages API dependencies.

5.2 Key Endpoints

The API provides endpoints for managing users, games, friends, and libraries. Below are examples of core functionalities:

- **User Management**
 - GET **/user/get_by_id/{id}** – Retrieves user information by ID.
 - POST **/user/create** – Creates a new user.
 - PUT **/user/update/{id}** – Updates user details.
 - DELETE **/user/delete/{id}** – Deletes a user.
- **Game Management**
 - GET **/game/get_by_id/{id}** – Fetches game details.
 - POST **/game/create** – Adds a new game to the database.
 - DELETE **/game/delete/{id}** – Removes a game from the system.
- **Library Management**
 - GET **/library/get-games-by-user-id/user-id** – Retrieves a user's game library.
 - POST **/library/create** – Adds a game to a user's library.
 - DELETE **/library/delete/user-id/game-id** – Removes a game from a user's library.
- **Friends System**
 - POST **/friends/create** – Adds a friend to a user's list.
 - DELETE **/friends/delete** – Removes a friend from user.

5.3 Why RESTful API?

The RESTful API architecture provides:

- **Scalability** – Supports multiple client interactions efficiently.
- **Statelessness** – Each request contains all necessary information, reducing server load.
- **Flexibility** – Can be extended to support more features in the future.

By integrating FastAPI with a structured database model and containerized deployment, the project ensures efficient and scalable API operations.

Conclusions

- It has been concluded that Steam relies heavily on games and users, meaning that the database model relationships usually aim to the users and the games.
- It was also concluded that the design of this model might be just a lite version of the actual database of steam due to it's complexity and size.
- It was concluded that for a better flexibility and scalability, it is best to use a RESTful API which in this case was developed with FastAPI. The API enables seamless interaction with the database, offering CRUD functionalities for users, games, and friendships.
- The API's implementation follows REST principles, ensuring stateless communication and ease of integration with various client applications which are very similar to the CRUD Operations in SQL.

References

- [1] Valve Corporation. **Valve Corporation: About Us**. <https://www.valvesoftware.com/es/about>, September 27th, 2021.